

Chat System — High-Level Design

1. Communication Protocol Choice

Earlier, we selected:

None

WebSocket

as the primary communication protocol because it supports:

- Bidirectional communication
- Persistent connections
- Real-time messaging

2. Important Clarification

Not every feature in the chat application needs WebSocket.

Many normal application features still use:

None

HTTP Request/Response

3. Features Using HTTP

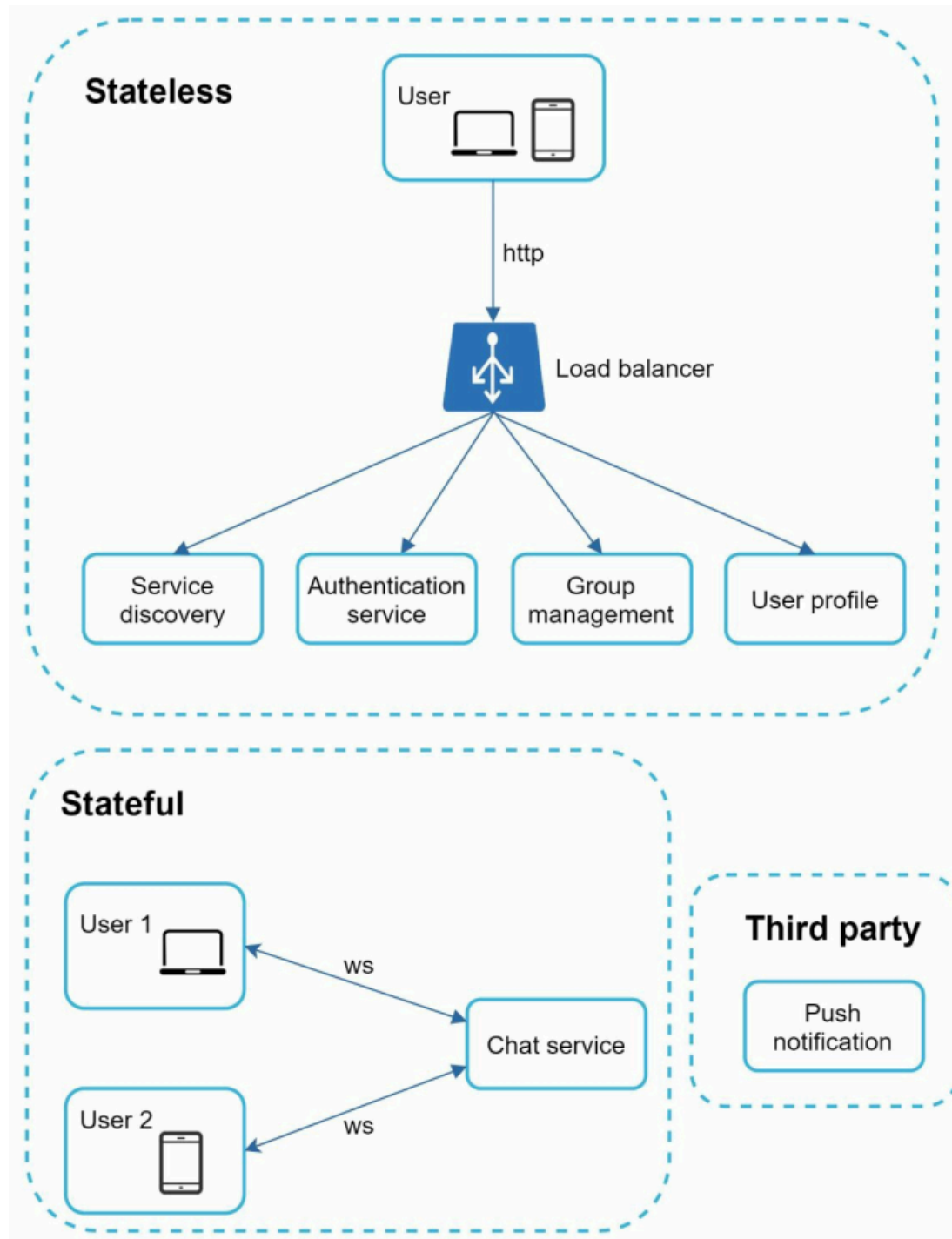
Traditional HTTP APIs are suitable for:

- User signup

- User login
- Profile management
- Friend management
- Settings update
- Search operations

These are standard stateless operations.

4. Major Components of Chat System



The chat system is divided into three major categories:

1. Stateless Services
2. Stateful Services
3. Third-Party Integration

5. Stateless Services

Stateless services are traditional public-facing services.

Responsibilities

They handle:

- Login
- Signup
- User profile
- Friend list
- Authentication
- Settings

Characteristics

These services:

- Do not maintain client session state
- Use request-response communication
- Can scale horizontally easily

Load Balancer Usage

Stateless services are placed behind:

None

Load Balancer

The load balancer:

- Distributes requests
- Routes traffic to correct services
- Improves scalability

Architecture Options

Stateless services can be implemented as:

- Monolithic architecture
OR
- Microservices architecture

6. Service Discovery

One important stateless service is:

None

Service Discovery

Purpose

Service discovery helps clients find available chat servers.

Main Function

It provides:

None

List of DNS hostnames of chat servers

that clients can connect to.

Why It Is Important

It helps:

- Balance traffic
- Avoid overloaded servers
- Improve availability

7. Stateful Services

The primary stateful service is:

None

Chat Service

8. Why Chat Service is Stateful

Each client maintains:

None

Persistent WebSocket Connection

with a chat server.

The server must continuously maintain connection state.

9. Important Behavior

Normally:

- A client stays connected to the same chat server
- Connection remains active as long as server is healthy

This avoids unnecessary reconnect operations.

10. Coordination with Service Discovery

Service discovery works closely with chat servers to:

- Monitor server load
- Route new users intelligently
- Prevent server overload

11. Third-Party Integration

The most important third-party integration is:

None

Push Notification Service

12. Why Push Notifications Are Important

Users may:

- Close app
- Go offline
- Lock devices

Push notifications inform them about:

- New messages
- Mentions
- Alerts

even when the app is inactive.

13. Examples of Push Notification Providers

Examples include:

- APNS (Apple Push Notification Service)
- FCM (Firebase Cloud Messaging)

14. Scalability Discussion

Small-Scale Scenario

At very small scale:

None

All services can fit on one server

Why This Is Possible

Even 1 million concurrent users may theoretically fit on one modern cloud server.

Example Estimation

Assume:

None

1 connection = 10 KB memory

For:

None

1 million concurrent users

Required memory:

None

≈ 10 GB RAM

15. Why Single Server is a Bad Idea

Although theoretically possible:

None

Single Server Design = Dangerous

Major Problems

Single Point of Failure (SPOF)

If server crashes:

- Entire system goes down

Poor Scalability

Scaling vertically has limits.

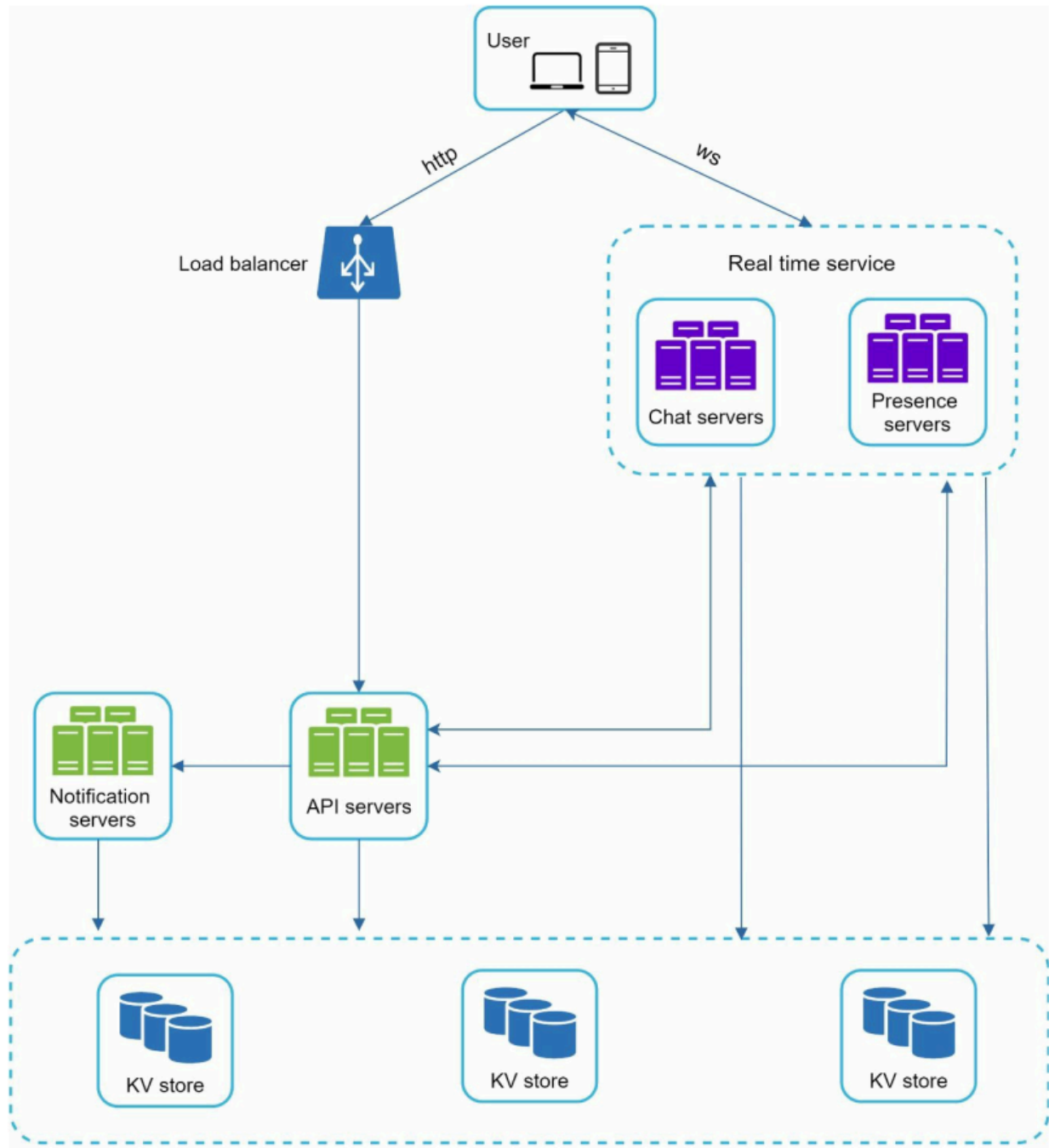
Maintenance Difficulties

Upgrades and deployments become risky.

Reliability Issues

Hardware or network failure affects all users.

16. Better Distributed Design



The improved architecture separates components into multiple servers.

17. Main Components in Final High-Level Design

Chat Servers

Responsibilities:

- Real-time messaging
- WebSocket management
- Message routing

Presence Servers

Manage:

- Online/offline status
- Last seen timestamps
- Presence updates

API Servers

Handle:

- Login
- Signup
- User profile
- Authentication
- General APIs

Notification Servers

Send:

- Push notifications
- Message alerts

Key-Value Store

Stores:

- Chat history
- Offline messages
- Conversation data

18. Offline Message Handling

If user is offline:

1. Messages are stored in key-value store
2. When user reconnects:
 - Previous chat history is fetched
 - Undelivered messages are shown

19. Storage Design

Choosing the correct database is critical.

20. Two Types of Data in Chat Systems

Type 1: Generic Application Data

Examples:

- User profile
- Settings
- Friend list

Suitable Storage

None

Relational Database (SQL)

Why SQL?

Because it provides:

- Strong consistency
- Structured schema
- Reliable transactions

Common Scaling Techniques

- Replication
- Sharding

21. Type 2: Chat History Data

This is unique to chat systems.

Characteristics of Chat Data

Massive Data Volume

Example:

None

Facebook Messenger + WhatsApp

≈ 60 billion messages/day

Recent Messages Accessed Frequently

Users mostly view:

- Recent chats
- Latest conversations

Random Access Support Needed

Features requiring random access:

- Search messages
- Mentions
- Jump to old messages

Read/Write Ratio

For one-on-one chat:

None

Read : Write ≈ 1 : 1

22. Why Key-Value Store is Preferred

The recommended storage solution is:

None

Key-Value Store

23. Advantages of Key-Value Stores

Easy Horizontal Scaling

Can distribute data across many servers.

Low Latency

Fast read/write operations.

Better for Large-Scale Chat Data

Handles massive data efficiently.

Efficient for Long-Tail Data

Unlike relational DBs:

- Large indexes become expensive
- Random access slows down

Key-value stores perform better at scale.

24. Real-World Examples

Popular chat applications use key-value stores.

Application	Storage Technology
Facebook Messenger	HBase
Discord	Cassandra

25. Final High-Level Architecture Summary

None

Client ↔ WebSocket ↔ Chat Servers



Presence Servers



API Servers



Notification Servers



Key-Value Store

Key Takeaway

Modern chat systems combine:

- HTTP for stateless APIs
- WebSocket for real-time communication

Architecture is divided into:

- Stateless services
- Stateful chat services
- Third-party integrations

Key-value stores are preferred for chat history because they provide:

- Low latency
- Massive scalability
- Efficient storage for huge messaging workloads

Distributed architecture eliminates:

None

Single Point of Failure

and improves scalability and reliability.